

PRAKTIKUM 4

Studi Kasus Dunia Nyata: Jaringan Komputer

1. Studi Kasus

NIM belakang 4 → Studi Kasus:

Jaringan Komputer

Studi kasus ini merepresentasikan sebuah topologi jaringan komputer kantor sederhana yang terdiri dari dua area jaringan (segmen) yang dihubungkan oleh sebuah router pusat. Setiap perangkat dianggap sebagai node (vertex) dan kabel fisik yang menghubungkan antar perangkat dianggap sebagai edge.

2. Penentuan Node dan Edge

2.1 Node (Vertex)

Terdapat 7 node dalam jaringan ini:

No.	Node	Keterangan
1	Router	Perangkat utama yang menghubungkan dua segmen jaringan
2	Switch1	Switch untuk area jaringan 1 (kiri)
3	Switch2	Switch untuk area jaringan 2 (kanan)
4	PC1	Komputer pengguna 1, terhubung ke Switch1
5	PC2	Komputer pengguna 2, terhubung ke Switch1
6	PC3	Komputer pengguna 3, terhubung ke Switch2
7	Server	Server pusat, terhubung ke Switch2

2.2 Edge (Koneksi Kabel)

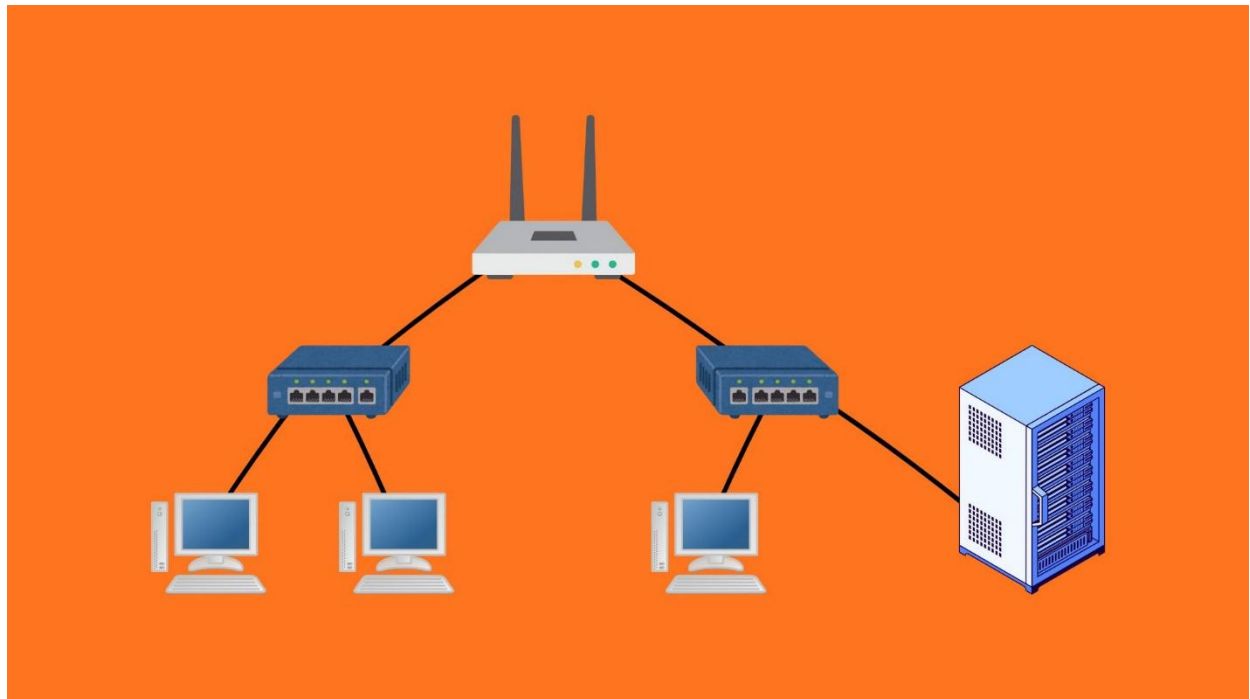
Terdapat 6 edge yang merepresentasikan koneksi kabel fisik:

No.	Node A	Node B	Keterangan
1	Router	Switch1	Koneksi router ke switch area 1

2	Router	Switch2	Koneksi router ke switch area 2
3	Switch1	PC1	Koneksi switch1 ke komputer PC1
4	Switch1	PC2	Koneksi switch1 ke komputer PC2
5	Switch2	PC3	Koneksi switch2 ke komputer PC3
6	Switch2	Server	Koneksi switch2 ke server pusat

3. Desain Graph (Topologi Jaringan)

Berikut representasi teks dari topologi jaringan yang dirancang:



4. Implementasi Python

4.1 Adjacency List

Adjacency list direpresentasikan menggunakan dictionary Python, di mana key adalah nama node dan value adalah list tetangga yang terhubung langsung:

```
# =====
# Praktikum 4 - Studi Kasus Dunia Nyata
# =====

graph = {
    "Router": ["Switch1", "Switch2"],
    "Switch1": ["Router", "PC1", "PC2"],
    "Switch2": ["Router", "PC3", "Server"],
    "PC1": ["Switch1"],
    "PC2": ["Switch1"],
    "PC3": ["Switch2"],
    "Server": ["Switch2"]
}

print('Adjacency List Representation:')
for node, neighbors in graph.items():
    print(f'{node}: {" -> ".join(neighbors)}')
```

4.2 Output Adjacency List

```
Adjacency List Representation:
Router : Switch1 -> Switch2
Switch1 : Router -> PC1 -> PC2
Switch2 : Router -> PC3 -> Server
PC1 : Switch1
PC2 : Switch1
PC3 : Switch2
Server : Switch2
```

4.3 Tabel Adjacency List

Node (Vertex)	Tetangga (Neighbour)
Router	Switch1, Switch2
Switch1	Router, PC1, PC2
Switch2	Router, PC3, Server
PC1	Switch1
PC2	Switch1
PC3	Switch2
Server	Switch2

4.4 Adjacency Matrix

Adjacency matrix menggunakan indeks numerik untuk setiap node. Nilai 1 menandakan ada koneksi, nilai 0 berarti tidak ada koneksi langsung:

```
Adjacency Matrix Representation:
-----
      Rout  Swit  Swit  PC1  PC2  PC3  Serv
-----
Router      0      1      1      0      0      0      0
Switch1     1      0      0      1      1      0      0
Switch2     1      0      0      0      0      1      1
PC1          0      1      0      0      0      0      0
PC2          0      1      0      0      0      0      0
PC3          0      0      1      0      0      0      0
Server       0      0      1      0      0      0      0
```

4.5 Tabel Adjacency Matrix

Sel berwarna biru menunjukkan nilai 1 (ada koneksi):

Node	Router	Switch	Switch	PC1	PC2	PC3	Server
Router	0	1	1	0	0	0	0
Switch1	1	0	0	1	1	0	0
Switch2	1	0	0	0	0	1	1
PC1	0	1	0	0	0	0	0
PC2	0	1	0	0	0	0	0
PC3	0	0	1	0	0	0	0
Server	0	0	1	0	0	0	0

5. Analisis

5.1 Jenis Graph

Graph jaringan komputer ini merupakan Undirected Graph (graph tidak berarah). Setiap koneksi kabel jaringan bersifat dua arah — jika PC1 dapat mengirim data ke Switch1, maka Switch1 juga dapat mengirim data kembali ke PC1. Tidak ada arah preferensi dalam komunikasi fisik jaringan ini.

5.2 Alasan Pemilihan Edge

Edge dipilih berdasarkan koneksi kabel fisik yang menghubungkan antar perangkat jaringan. Pertimbangan pemilihan:

- Setiap perangkat hanya terhubung ke switch atau router terdekat sesuai topologi hierarkis.
- Router sebagai gateway pusat menghubungkan dua segmen jaringan (Switch1 dan Switch2).
- PC dan Server hanya terhubung ke switch masing-masing, bukan langsung ke router.
- Tidak ada koneksi langsung antar PC untuk menjaga efisiensi topologi.

5.3 Dense atau Sparse?

Graph ini tergolong SPARSE GRAPH (graph jarang):

- Jumlah node (V) = 7
- Jumlah edge aktual (E) = 6
- Jumlah edge maksimum untuk undirected graph = $V*(V-1)/2 = 7*6/2 = 21$
- Densitas = $6/21 \approx 28.6\% \rightarrow$ jauh dari padat (dense biasanya $> 70\%$)

5.4 Perbandingan Representasi

Aspek	Adjacency Matrix	Adjacency List
Bentuk	Matriks 2D ($V \times V$)	Dictionary / List
Kompleksitas Ruang	$O(V^2) = 49$ sel	$O(V+E) = 13$ elemen
Memori	Boros (banyak nilai 0)	Hemat (hanya simpan edge)
Cek Edge	Sangat cepat $O(1)$	Lebih lambat $O(\text{degree})$
Cocok untuk	Dense graph	Sparse graph ✓
Mudah dipahami	Ya	Sedang
Rekomendasi	Tidak ideal untuk kasus ini	✓ Lebih cocok

5.5 Representasi yang Lebih Cocok

Adjacency List jauh lebih cocok untuk kasus ini karena:

- Graph bersifat sparse sehingga adjacency matrix akan menyimpan banyak nilai 0 yang tidak berguna.
- Adjacency matrix membutuhkan $7 \times 7 = 49$ sel, sementara hanya 12 sel bernilai 1 (efisiensi hanya $\sim 24\%$).
- Adjacency list hanya menyimpan koneksi yang nyata, sehingga lebih hemat memori dengan kompleksitas $O(V+E) = O(13)$.
- Untuk traversal BFS/DFS pada pertemuan berikutnya, adjacency list memudahkan iterasi tetangga suatu node.

6. Kesimpulan

Praktikum ini berhasil merepresentasikan topologi jaringan komputer sederhana menggunakan struktur data graph dalam Python. Beberapa poin penting yang dapat disimpulkan:

- Graph jaringan komputer ini adalah undirected graph karena komunikasi jaringan bersifat dua arah.
- Topologi yang digunakan adalah topologi hierarkis (bertingkat): Router $\rightarrow$ Switch $\rightarrow$ End Device.
- Dengan 7 node dan 6 edge, graph ini bersifat sparse sehingga adjacency list adalah representasi yang paling efisien.
- Adjacency matrix tetap berguna jika sering dilakukan pengecekan koneksi langsung antar dua perangkat ($O(1)$).
- Representasi graph yang telah dibuat ini siap digunakan sebagai input algoritma traversal BFS dan DFS pada pertemuan selanjutnya, misalnya untuk mencari jalur terpendek antar perangkat dalam jaringan.